GIT & GITHUB
PRACTICAL MODULE 1

Connect an existing vibe-coded project to GitHub

Instructor-led, step-by-step classroom guide

Concepts • Terminal commands • Security checks • Troubleshooting • Student checklist

Learning objective

By the end of this practical, each student will be able to take code they have already written in Windsurf, Devin, or another IDE, track it with Git, create a safe initial commit, connect it to a GitHub repository, and push it successfully.

Recommended duration: 30–40 minutes. This guide assumes students already have a project folder and a GitHub account.

Important: command examples use placeholders such as USERNAME and REPO. Students must replace them with their own values.

1. The concepts students must understand

Start with the mental model. Git commands make much more sense once students understand where files and commits live.

Git vs GitHub

Concept	Meaning	Simple analogy
Git	Version-control software that records changes and commits on a computer.	A timeline of snapshots of your project.
GitHub	A website/platform that hosts Git repositories and supports collaboration, pull requests, reviews, and issues.	A shared online workspace for the team.
Repository (repo)	The project files plus Git history. A local repo is on your computer; a remote repo is hosted elsewhere.	The project folder plus its history.
Remote	A named connection to another repository, commonly hosted on GitHub.	A saved address for the online copy.
origin	The conventional name Git gives to the primary remote. It is just a nickname.	A contact name for the remote repository.

The four places changes move through

Stage	What it means	Command / action
Working directory	Files as they currently exist on disk; edits may be untracked or unstaged.	Edit and save files in the IDE.
Staging area / index	The exact changes selected for the next commit.	git add
Local repository	Committed snapshots stored in local Git history.	git commit
Remote repository	Commits shared with GitHub or another remote.	git push

Key teaching line: Saving a file, staging a change, committing it, and pushing it are four different actions.

The common first-time flow

```
edit files → git add → git commit → git push
```

A commit is local until it is pushed. Pushing does not automatically create a pull request or merge code into the main branch.

2. Step-by-step practical

Step 1 — Open the correct project folder

Open the folder containing the app students already built. Do not accidentally initialize Git in the parent folder or a different project.

```
pwd
# On Windows PowerShell, this also works:
Get-Location
```

Expected result: the terminal displays the path to the project folder. If it is not the correct folder, navigate to the correct one before continuing.

Step 2 — Check that Git is installed

```
git --version
```

If a version is printed, Git is available. If the command is not found, install Git from git-scm.com/downloads, restart the terminal, and retry.

Step 3 — Configure the commit identity

Git records an author name and email on commits. This is commit metadata; it is not the same thing as logging into GitHub.

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"

# Verify the settings
git config --global user.name
git config --global user.email
```

What --global means: these settings apply to the user's Git configuration across repositories on that computer. Without --global, a setting can be configured only for the current repository.

Step 4 — Check whether this project is already a Git repository

```
git status
git remote -v
```

If git status shows a branch and file changes, Git is already initialized. Do not initialize it again or replace an existing remote without understanding it. If Git says this is not a repository, and the folder is a new local project, continue to the next step.

3. Protect files before staging

Step 5 — Create a .gitignore file

Create a file named .gitignore in the project root. It tells Git which untracked files to ignore. This is especially important for AI projects, which may contain API keys, local environments, model files, generated data, and logs.

```
# Secrets / environment variables
.env
.env.*
!.env.example

# Python virtual environments
.venv/
venv/
env/

# Python-generated files
__pycache__/
*.py[cod]
.pytest_cache/
.mypy_cache/

# OS/editor files
.DS_Store
Thumbs.db

# Logs
*.log
```

This is a starting template, not a universal policy. Review the project before ignoring data, models, or outputs; small sample data or configuration files may be required by the project.

Critical security note: .gitignore does not remove a file that Git already tracks, and it does not erase a secret from existing commit history. If a credential has been exposed, revoke/rotate it and follow the organisation's incident process.

Before staging, visually inspect the project for .env files, API keys, access tokens, passwords, private data, large generated files, and local database files. Never upload secrets or private customer data.

Step 6 — Initialize Git if needed

Use this only if git status previously confirmed that the folder is not already a repository.

```
git init -b main
```

This creates the local Git metadata and a main branch; it does not upload anything. If your Git version does not support -b main, use the fallback:

```
git init
git branch -M main
```

Step 7 — Inspect the repository state

```
git status
```

Git status is the safest first diagnostic command. It tells you the current branch and whether files are untracked, modified, staged, or clean.

4. Add, stage, commit: what each means

Step 8 — Stage the changes

The staging area is a preview of what will go into the next commit. Staging lets a developer select some changes while leaving other edits out of that commit.

```
git add .
git status
git diff --cached
```

git add . stages changes under the current directory. git diff --cached displays the staged content, so students can inspect what they are about to save. Prefer staging individual files when appropriate:

```
git add app.py
```

If a file was staged by mistake, unstage it without deleting it from the computer:

```
git restore --staged filename
```

Replace filename with the actual path. Then check git status again.

Step 9 — Create the first commit

A commit is a named snapshot of the staged changes in the local repository. A commit message should briefly describe what the snapshot changes.

```
git commit -m "Initial commit"
git log --oneline
```

For a first commit, Initial commit is fine. Later, use messages such as Add research assistant API or Add planner unit tests. The short hash shown by git log identifies the commit.

What is stash?

git stash temporarily shelves tracked working changes so you can switch tasks or branches without making a commit. It is useful when you have unfinished work that is not ready to commit. It is not a replacement for commits or a reliable backup.

```
git stash push -m "work in progress"
git stash list
git stash pop
```

stash pop reapplies the latest stash and removes it from the stash list if it applies successfully. Conflicts can occur. By default, ordinary stash usage may not include untracked files; use git stash push -u when you also need to stash untracked files. Do not stash secrets as a way to make them safe.

5. Create the GitHub repository and push

Step 10 — Create an empty repository on GitHub

Open github.com/new, sign in, choose a repository name such as ai-research-assistant, and select Private for classroom work that should not be public. For this local-first workflow, leave README, .gitignore, and license initialization unchecked so the remote starts empty.

Step 11 — Connect the local repo to GitHub

Copy the HTTPS URL from the GitHub repository page. Replace the example URL below with the actual one.

```
git remote add origin https://github.com/USERNAME/REPO.git
git remote -v
```

git remote add creates a named remote connection. origin is a conventional nickname. git remote -v should show the URL for both fetch and push.

If Git reports that origin already exists, do not remove it blindly. Run git remote -v and confirm whether it points to the intended repository.

Step 12 — Push the first commit

```
git push -u origin main
```

git push uploads local commits to the remote. origin identifies the remote and main is the branch to upload. -u sets upstream tracking, so future pushes from that branch can usually use git push without repeating the remote and branch.

If authentication is requested, use GitHub's supported sign-in flow or an approved credential manager/token method. GitHub does not accept an account password for HTTPS Git operations.

Step 13 — Verify the result

```
git status
git log --oneline
git remote -v
```

Refresh the repository page on GitHub. The files and commit should be visible. A successful push normally leaves the local branch tracking origin/main and the working tree clean.

6. Command cheat sheet

Command	Meaning / when to use it
git --version	Check that Git is installed.
git status	Inspect current branch and working-tree state; run it often.
git init -b main	Initialize a new local repository with main as the initial branch.
git add file.py	Stage a specific file for the next commit.
git add .	Stage changes under the current directory; inspect first.
git diff	Show unstaged changes.
git diff --cached	Show staged changes.
git restore --staged file	Unstage a file without deleting its contents.
git commit -m "message"	Record the staged snapshot locally.
git log --oneline	Show compact commit history.
git stash push -m "WIP"	Temporarily shelve tracked working changes.
git stash list	List saved stashes.
git stash pop	Reapply the latest stash; conflicts are possible.
git remote -v	Show configured remote URLs.
git remote add origin URL	Connect local repo to the GitHub remote.
git push -u origin main	Upload main and set upstream tracking.

Everyday mental model

```
edit → git status → git add → git diff --cached
     → git commit → git push
```

Never treat a successful commit as proof that code has been uploaded. The push is the upload step.

7. Common errors and fixes

Error / symptom	Likely cause	Safe next step
git: command not found	Git missing or terminal PATH not updated.	Install Git, restart terminal, retry git --version.
Author identity unknown	Name/email not configured.	Set user.name and user.email.
origin already exists	A remote is already configured.	Inspect git remote -v before changing it.
Authentication failed	Login/permissions issue.	Use approved GitHub authentication; confirm repo access.
src refspec main does not match any	No commit yet or branch has a different name.	Check git status and git branch.
rejected (fetch first)	Remote has commits not in local history.	Inspect both histories before pulling or reconciling.
.env is staged	Secret file was not ignored or is already tracked.	Unstage it; verify tracking/history; rotate exposed secrets.

8. Corporate safety habits

AI coding tools can generate changes quickly, but the developer remains responsible for what is committed and shared.

- Never commit API keys, tokens, passwords, private certificates, or customer data.
- Use .env.example for placeholder configuration; never put real credentials in it.
- Review git status and git diff --cached before every commit.
- Do not assume .gitignore removes already tracked files or erases Git history.
- Avoid committing virtual environments, caches, logs, and large generated artifacts unless the project specifically requires them.
- Do not blindly run commands suggested by an AI agent; understand destructive or history-changing commands first.
- Use a private repository for classroom projects unless public sharing is intentional and safe.

9. Student completion checklist

■ Correct project folder opened in the IDE terminal

■ Git installed and name/email configured

■ Existing repository status checked before initializing

■ .gitignore created and sensitive files reviewed

■ Local Git repository initialized if needed

■ Staged changes inspected with git diff --cached

■ Initial commit created and visible in git log

■ Empty GitHub repository created

■ Remote URL configured and verified

■ git push -u origin main completed successfully

■ Files and commit visible on GitHub

10. Official references

- Git documentation: git-scm.com/doc
- GitHub: adding locally hosted code: GitHub Docs
- GitHub authentication: GitHub Docs
- Pro Git book (free): git-scm.com/book/en/v2

Instructor's transition to Module 2: Once students have uploaded their own projects, have them clone an unfamiliar repository and analyse its structure before changing any code. That prepares them for branch-based corporate collaboration.